

HCODE · MASTER CLASS · 4 HORAS · LAB GUIADO

Agentes no ambiente de desenvolvimento

GitHub Copilot · .github · .vscode

Claude Code · .claude · CLAUDE.md · AGENTS.md

Glaucio Daniel · CEO, Hcode | Lab: github.com/Hcode-AI-Team/aula-1-combio-manutencao

ROTEIRO

Agenda das 4 horas

Horário	Bloco	Min
00:00	Abertura e setup do lab (Missão 0)	15
00:15	Fundamentos: contexto, primitivas, escopos	25
00:40	GitHub Copilot: .github e .vscode	25
01:05	Lab A — Copilot (Missões 1 a 4)	45
01:50	Intervalo	10
02:00	Claude Code: .claude, CLAUDE.md, AGENTS.md	25
02:25	Lab B — Claude Code (Missões 5 a 8)	40
03:05	Lab C — a feature ponta a ponta nas duas ferramentas	40
03:45	Convergência, segurança e encerramento	15

O laboratório: um sistema real de manutenção de usinas

combio-manutencao

- Ordens de manutenção das UPVs (usinas de biomassa)
- Backend NestJS 11 + TypeORM + SQLite
- Frontend Angular standalone
- 93 testes unitários + 25 e2e, CI no GitHub Actions
- CLAUDE.md com títulos vazios e .claude/agents/ vazio — de propósito

A missão do dia

- Hoje uma ordem **cancelada** pode voltar a **aberta**
- `updateStatus()` sobrescreve `concluidaEm`; `update()` preserva
- Um teste documenta essa assimetria como "não desejável"
- Vamos entregar a máquina de estados — com agentes que nós mesmos configuramos

01

Fundamentos

O que muda quando o agente passa a ler arquivos do seu repositório — e por que isso é engenharia, não prompt.

A TESE DA AULA

O modelo não conhece o seu repositório. Customização é o contexto que sobrevive ao prompt.

Arquivo versionado vale para o time inteiro, passa por code review e não depende da memória de quem digitou o prompt.

Sete primitivas, uma pergunta: quem ativa?

Primitiva	Use quando...	Quem ativa
Instruções sempre ativas	o padrão vale para o projeto inteiro	automático, em toda requisição
Instruções por caminho	a regra vale só para uma pasta ou tipo de arquivo	arquivo que casa com o glob
Prompt / command	uma tarefa se repete e você quer disparar	você, com /nome
Skill	um procedimento precisa de passos e arquivos de apoio	o agente (pela description) ou /nome
Agente / subagente	um papel precisa de ferramentas e modelo próprios	você escolhe, ou outro agente delega
MCP	o agente precisa falar com um sistema externo	o agente chama a ferramenta
Hook	uma ação não pode depender da vontade do modelo	o evento do ciclo do agente

Instrução é pedido. Hook é garantia.

Guia o modelo — probabilístico

- Instruções, skills, prompts, agentes
- O modelo lê, interpreta e decide se segue
- Ótimo para estilo, arquitetura, domínio
- Falha em silêncio quando o contexto fica cheio ou as regras conflitam

Executa sempre — determinístico

- Hooks (PreToolUse, PostToolUse...)
- Permissões allow / ask / deny
- Settings gerenciados pela empresa
- Roda quando o evento acontece, lembre o modelo ou não

Regra prática: se não pode falhar, não é instrução — é hook ou permissão.

ESCOPOS

Escolha o escopo mais estreito que resolve

Usuário

- Preferências pessoais
- ~/.copilot/ e ~/.claude/
- CLAUDE.local.md no projeto
- Não vai para o git

Repositório

- Arquitetura, comandos, domínio
- AGENTS.md, CLAUDE.md
- .github/, .vscode/, .claude/
- Versionado e revisado em PR

Organização

- Política e compliance
- Custom agents da org no GitHub
- CLAUDE.md e settings gerenciados
- Não pode ser desligado pelo dev

02

GitHub Copilot no VS Code

Instruções, prompt files, custom agents, subagentes, skills e hooks — tudo em `.github/` e `.vscode/`.

Onde cada coisa mora no Copilot

```
AGENTS.md                sempre ativo
.github/
  copilot-instructions.md  sempre ativo
  instructions/*.instructions.md  por applyTo
  prompts/*.prompt.md      /comando
  agents/*.agent.md         papéis
  skills/<nome>/SKILL.md    skills
  hooks/*.json              hooks
.vscode/
  settings.json             liga os recursos
  mcp.json                  servidores MCP
```

Também lê o formato Claude

CLAUDE.md, .claude/agents/, .claude/skills/ e os hooks de .claude/settings.json.

Na dúvida: Diagnostics

Botão direito no Chat → Diagnostics lista tudo o que carregou e os erros de frontmatter.

INSTRUCTIONS

Instruções: sempre ativas e por caminho

```
# .github/instructions/backend-nestjs.instructions.md
---
applyTo: "backend/**/*.ts"
description: "Padrões NestJS + TypeORM"
---
# Backend NestJS
- Repositórios com @InjectRepository
- Relações explícitas; nada de eager
- Conflito de regra → ConflictException (409)
```

- **copilot-instructions.md** e **AGENTS.md**: entram em toda requisição
- ***.instructions.md**: entra quando o arquivo casa com [applyTo](#) ou a tarefa casa com a [description](#)
- [/init](#) gera as sempre ativas; [/create-instruction](#) gera as por caminho
- Peça "extraia uma instrução disso" depois de corrigir o agente

PROMPTS

Prompt files: o slash command do time

```
# .github/prompts/nova-feature.prompt.md
---
description: "Planeja e implementa com TDD"
agent: agent
argument-hint: "Descreva a feature"
tools: ['search', 'read', 'edit', 'execute']
---
Feature: ${input:feature}
1. Leia AGENTS.md e os arquivos envolvidos
2. Escreva os testes que falham
3. Implemente o mínimo e rode os testes
4. Rode o lint e resuma o que mudou
```

- Invocado com `/nova-feature` no Chat
- `${input:...}` pede o argumento na hora
- `tools` do prompt têm prioridade sobre as do agente
- Use para tarefas curtas e repetidas; procedimento com arquivos de apoio vira skill

Custom agents: papel + ferramentas + modelo

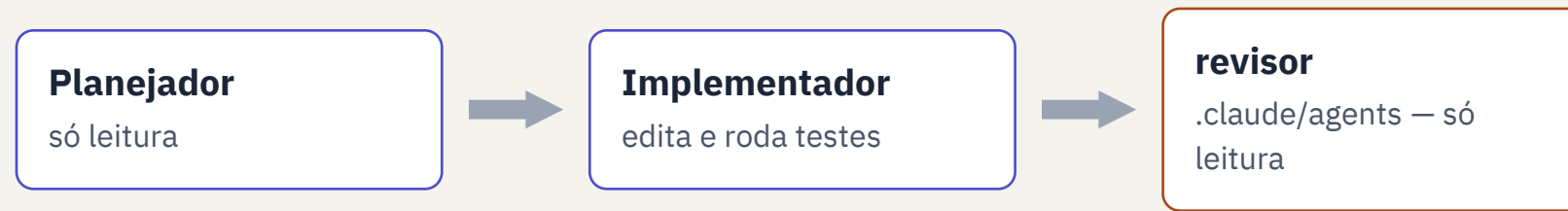
```
# .github/agents/planejador.agent.md
---
name: Planejador
description: Plano sem editar arquivos
tools: ['search', 'read', 'web/fetch', 'todo']
handoffs:
  - label: Implementar o plano
    agent: Implementador
    prompt: Implemente o plano com TDD
---
Você é o planejador. Não edite arquivos.
Entregue: objetivo, arquivos, testes, riscos.
```

- **tools**: o menor conjunto possível — planejador não tem **edit**
- **model**: um nome ou lista com fallback
- **agents**: quem ele pode chamar como subagente
- **user-invocable** / **disable-model-invocation**: visível no seletor × chamável por outros
- Antigos **.chatmode.md** → renomeie para **.agent.md**

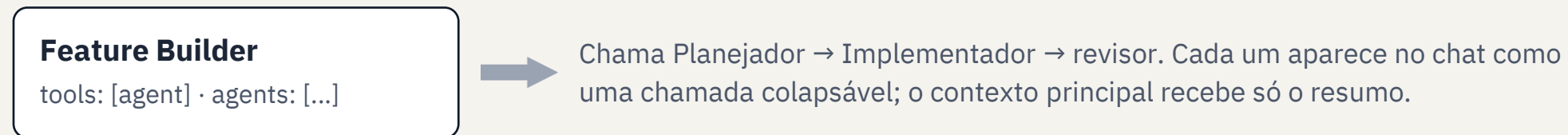
ORQUESTRAÇÃO

Handoff deixa você no controle; subagente delega

HANDOFF — botão ao fim da resposta, você revisa antes de seguir



SUBAGENTE — o agente chama outro com contexto isolado e recebe só o resultado



Skills: procedimento que só entra quando precisa

```
.claude/skills/regras-ordem-manutencao/  
SKILL.md
```

```
---
```

```
name: regras-ordem-manutencao
```

```
description: Regras do ciclo de vida  
da ordem. Use ao alterar status,  
datas ou validação de ordens.
```

```
---
```

```
| De \ Para | aberta | em_execucao | ...  
Transição proibida → 409 Conflict
```

- Padrão aberto Agent Skills: a mesma pasta funciona em várias ferramentas
- O agente vê só `name` + `description`; o corpo carrega quando a tarefa casa
- Pasta pode ter scripts, templates e exemplos
- Copilot lê `.github/skills`, `.claude/skills` e `.agents/skills`
- Claude Code lê `.claude/skills` — por isso é o lugar compartilhado

.vscode: ligar recursos e conectar sistemas

```
// .vscode/settings.json
{
  "chat.useAgentsMdFile": true,
  "chat.useClaudeMdFile": false,
  "chat.useHooks": true,
  "chat.agentSkillsLocations": {
    ".claude/skills": true
  }
}
```

```
// .vscode/mcp.json
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

Hooks do Copilot (Preview) ficam em [.github/hooks/*.json](#) ou são lidos de [.claude/settings.json](#). Confirme os nomes das chaves na sua versão em Diagnostics.

Lab A — Copilot

Missão	O que criar	Pronto quando
M1 · 15 min	AGENTS.md, copilot-instructions.md, instructions/ backend e frontend	a pergunta da M0 cita 409 e os status certos
M2 · 10 min	prompts/nova-feature.prompt.md	/nova-feature aparece com o argument-hint
M3 · 5 min	skills/regras-ordem-manutencao em .github/skills	"posso reabrir uma ordem cancelada?" carrega a skill
M4 · 15 min	agents/ Planejador, Implementador, Feature Builder + .vscode/settings.json	o Planejador planeja sem editar e mostra o handoff

Intervalo • 10 min

Na volta: abram um terminal na raiz do repositório e rodem **claude**. Quem não fez login ainda, faça agora.

03

Claude Code

Um agente no terminal que lê CLAUDE.md, obedece .claude/settings.json e delega para subagentes.

Claude Code em um minuto

O loop

- Lê o código e o contexto
- Planeja
- Edita arquivos
- Roda comandos e testes
- Verifica e itera

Modos (Shift+Tab)

- Manual: pede permissão
- Aceitar edições automaticamente
- Plan mode: só lê e planeja
- Onde roda: terminal, IDE, desktop, web

Comandos que vamos usar

- `/init` gera o CLAUDE.md
- `/context` mostra o que carregou
- `/memory` e `/config`
- `/agents` lista subagentes

Onde cada coisa mora no Claude Code

```
CLAUDE.md           time, versionado
CLAUDE.local.md      pessoal, .gitignore
.mcp.json            MCP do projeto
.claude/
  settings.json       permissões, hooks
  settings.local.json pessoal
  rules/*.md           por paths
  skills/<nome>/SKILL.md /nome
  commands/*.md        /nome (antigo)
  agents/*.md          subagentes
~/.claude/           o mesmo, do usuário
```

Ordem de carga

Gerenciado pela empresa → usuário → projeto → local.
Tudo é concatenado: nada sobrescreve nada.

Confira sempre

`/context` lista os memory files carregados na sessão.

CLAUDE.md: o que escrever e como carrega

Escopo	Local	Compartilhado com
Gerenciado	/etc/claude-code/CLAUDE.md (ou equivalente no SO)	toda a organização
Usuário	~/.claude/CLAUDE.md	só você, todos os projetos
Projeto	./CLAUDE.md ou ~/.claude/CLAUDE.md	o time, via git
Local	./CLAUDE.local.md	só você, este projeto

- Pastas acima carregam no início; subpastas, quando o Claude lê arquivos nelas
- @caminho importa outro arquivo (até 4 saltos) — organiza, mas **não economiza** contexto
- Mire em menos de 200 linhas; comentários HTML são removidos antes de entrar no contexto

AGENTS.md no Claude Code: quando ele é lido

O repositório tem...	O Claude lê
AGENTS.md e nenhum CLAUDE.md / CLAUDE.local.md	o AGENTS.md
AGENTS.md e também um CLAUDE.md	só o CLAUDE.md
CLAUDE.md que importa @AGENTS.md	os dois, sem duplicar

Padrão recomendado

CLAUDE.md = @AGENTS.md
+ só o que é do Claude

Detalhes que pegam

- Leitura direta exige v2.1.277+
- Bedrock ou telemetria desligada: use o import
- Mude em /config → Project instructions

Rules, skills e commands

```
# .claude/rules/backend-nestjs.md
---
paths:
  - "backend/**/*.ts"
---
# Backend NestJS
- Conflito de regra → 409
```

```
# .claude/commands/checar.md
---
description: Lint e testes
argument-hint: "[unit|e2e|all]"
allowed-tools: Bash(npm test:*)
---
Escopo: $ARGUMENTS
```

- Rule com `paths` só entra quando o Claude lê um arquivo que casa — igual ao `applyTo` do Copilot
- Commands foram fundidos em skills: `commands/checar.md` e `skills/checar/SKILL.md` geram o mesmo `/checar`
- Skill = pasta com arquivos de apoio e carga automática pela description

SUBAGENTS

Subagentes: contexto isolado, resposta resumida

```
# .claude/agents/revisor.md
---
name: revisor
description: Revisor de código e
  segurança. Use PROATIVAMENTE
  depois de mudanças no código.
tools: Read, Grep, Glob
model: sonnet
---
Você só lê; nunca edita.
Responda em tabela de severidade.
```

- `tools` / `disallowedTools`: o que pode usar
- `model`: sonnet, opus, haiku ou inherit
- `permissionMode`, `maxTurns`, `skills`, `hooks`
- `memory`: memória própria do subagente
- `isolation: worktree`: cópia isolada do repo
- O VS Code lê esta mesma pasta

settings.json: permissões e hooks

```
{
  "permissions": {
    "allow": ["Bash(npm test:*)"],
    "ask": ["Bash(git commit:*)"],
    "deny": ["Read(./env)",
             "Bash(git push:*)"]
  },
  "hooks": {
    "PreToolUse": [{ "matcher": "Read|Bash",
                      "hooks": [{ "type": "command",
                                   "command": "node .claude/hooks/protege-segredos.mjs"}]
    }]
  }
}
```

- **PreToolUse** pode bloquear a ferramenta
- **PostToolUse**: formatar, lintar, registrar
- **SessionStart, UserPromptSubmit, Stop, SubagentStop...**
- O hook recebe JSON no stdin e responde por exit code ou JSON
- O VS Code lê este arquivo, mas ignora o [matcher](#)

Lab B — Claude Code

Missão	O que criar	Pronto quando
M5 · 10 min	CLAUDE.md com @AGENTS.md, rules/backend-nestjs.md, CLAUDE.local.md	/context lista os três
M6 · 10 min	mover a skill para .claude/skills, skill testes-nestjs, commands/checar.md	/regras-ordem-manutencao e /checar funcionam nas duas ferramentas
M7 · 10 min	agents/revisor.md e agents/executor-testes.md	/agents lista os dois e o revisor aparece no VS Code
M8 · 10 min	settings.json + hooks/protege-segredos.mjs + hooks/lint-arquivo.mjs	os 4 testes de ataque são bloqueados

04

Convergência

Uma configuração, duas ferramentas: o que dá para compartilhar, o que não dá, e como proteger o repositório.

Matriz de equivalência

Primitiva	Copilot (VS Code)	Claude Code	Compartilhe em
Sempre ativa	copilot-instructions.md	CLAUDE.md	AGENTS.md
Por caminho	*.instructions.md · applyTo	.claude/rules · paths	— (formatos diferem)
Comando	.github/prompts/*.prompt.md	.claude/commands ou skills	—
Skill	.github/skills	.claude/skills	.claude/skills
Agente	.github/agents/*.agent.md	.claude/agents/*.md	.claude/agents
MCP	.vscode/mcp.json · servers	.mcp.json · mcpServers	— (chaves diferem)
Hook	.github/hooks/*.json	.claude/settings.json	.claude/settings.json

Estratégia de fonte única

1 • AGENTS.md é a verdade

CLAUDE.md = @AGENTS.md + específicos do Claude. copilot-instructions.md = só o que é do Copilot.

2 • Skills e agentes em .claude/

As duas ferramentas leem .claude/skills e .claude/agents. Formato Claude para agente compartilhado.

3 • Hooks em .claude/settings.json

Scripts Node que filtram o evento sozinhos, porque o VS Code ignora o matcher.

4 • Sem carga dupla no Copilot

chat.useClaudeMdFile: false — o AGENTS.md já entra; o CLAUDE.md repetiria o conteúdo.

Tudo o que o agente lê é dado, não ordem

Injeção por arquivo

- Docs, issues, comentários e dependências podem conter instruções escondidas
- Revise PRs que mexem em AGENTS.md, CLAUDE.md, .github/ e .claude/ como código
- Imports externos pedem aprovação — leia antes de aceitar

Menor privilégio

- Planejador e revisor sem edit
- deny para .env, *.sqlite e git push
- ask para commit e npm install
- Workspace trust ligado

Camada garantida

- PreToolUse bloqueia segredo antes de acontecer
- Revisor lista instruções suspeitas que encontrar
- Humano aprova o PR — sempre

Lab C — a feature, ponta a ponta

Critérios de aceite

1. aberta → em_execucao | concluida | cancelada
2. em_execucao → aberta | concluida | cancelada
3. concluida e cancelada são finais; mesmo status é idempotente
4. Transição proibida → 409 com mensagem padrão
5. update() e updateStatus() usam a mesma função
6. concluidaEm nunca é sobrescrita; testes e lint verdes

Rodada 1 · Copilot · 20 min

Feature Builder orquestra Planejador → Implementador → revisor.

Rodada 2 · Claude Code · 20 min

Plan mode → implementar com hook de lint → subagente revisor → /checar all.

CHECKLIST DE REVISÃO

Antipadrões que vimos em campo

Sintoma	Correção
CLAUDE.md ou AGENTS.md com centenas de linhas	rules com paths + skills; mire em menos de 200 linhas
"Sempre rode os testes" como instrução	hook ou subagente executor de testes
A mesma regra em três arquivos	AGENTS.md como fonte; os outros importam ou apontam
Agente com todas as ferramentas	lista mínima por papel
Skill com description vaga	diga o que cobre e quando usar
"Revise tudo duas vezes" nas instruções	modelos atuais já verificam; isso só gasta turnos

NA SEGUNDA-FEIRA, NO SEU REPOSITÓRIO

1

Um AGENTS.md curto

Comandos, domínio e o que nunca fazer. CLAUDE.md importando.

2

Um hook de segredo

PreToolUse bloqueando .env e push, versionado para o time.

3

Um agente revisor

Só leitura, em .claude/agents, usado pelas duas ferramentas.

Perguntas · Kit do lab e gabarito com o instrutor · Glaucio Daniel, Hcode